

1 Preliminaries

This document describes the implementation of a purely functional probabilistically balanced search tree. In this initial section, we define some basic terminology and notation that will be used throughout the document.

A tree structure T is defined as data structure accessed beginning at the root node. Each node is either a leaf or an internal node. An internal node has one or more child nodes and is called the parent of its child nodes. All children of the same node are siblings. Formally, a tree T is either empty (i.e., *null*), or a root and zero or more subtrees, where each subtree is a tree in its own right.

Given a total order $\preceq$ over a universe U , a tree structure T is defined as a *search tree* with respect to $\preceq$ if every subtree of a node has keys less than any other subtree of the node to its right. The keys in a node are conceptually between subtrees and are greater than any keys in subtrees to its left and less than any keys in subtrees to its right. This structure ensures that each key within the tree is unique and that the paths from the root to the leaves maintain a consistent hierarchical ordering according to $\preceq$, thereby facilitating efficient search, insertion, and deletion operations.

A *balanced tree* is defined by the property that no leaf node is significantly further from the root node than any other leaf node. The formal definition and enforcement of this balance can vary across different types of balanced trees, each characterized by specific rules or algorithms to maintain the desired level of balance.

In contrast to strictly balanced trees, a *probabilistically* balanced tree relaxes the deterministic constraints on balance, instead ensuring that the tree remains balanced with a certain probability. This approach can offer advantages in terms of reduced maintenance cost for the balance property during insertions and deletions, at the expense of a non-guaranteed but highly probable balance.

2 Zip trees

A zip tree is a probabilistically balanced binary search tree in which each node is augmented with a numeric rank. The rank is a non-negative integer that is chosen randomly from a geometric distribution with parameter $p = 1/2$. The tree is structured such that the rank of a node is greater than the rank of its left child and greater than or equal to the rank of its right child. In other words, the tree is (max)-heap-ordered with respect to ranks, with ties broken in favor of smaller keys.

We define a zip tree recursively as either an empty tree, or a non-empty tree such that it has a root $node = (key, rank, left, right)$ represented as a (named) tuple, with $key \in U$, $rank \in \mathbb{N}$ and two children $left$ and $right$ which are both valid trees. Additionally, each node must maintain the following balance invariants: for any non-empty tree T with root $node$, we have $\forall node' \in node.left \mid node'.key < node.key$ and $\forall node' \in node.right \mid node'.key > node.key$, and each $node \in T$ has a *rank* that satisfies $\forall node' \in node.left : node'.rank <$

$node.rank$ and $\forall node' \in node.right \mid node'.rank \leq node.rank$.

Let $S = \{s_1, s_2, \dots, s_n\}$ be a sorted set, where each element $s_i = (key, rank)$ has an associated *key* and *rank* as defined previously. The ordering of elements in S is determined based on keys, such that if $i < j$ then $S_i.key \preceq S_j.key$, where $\preceq$ denotes the total order relation on the keys. Using the above recursive definition for a zip tree, we can define a simple function (see alg. 1) that maps S to a zip tree T .

We define the maximum rank as $M_n = \max\{s.rank \mid s \in S\}$. Then, x is the element in S whose index is $i = \min\{i \in \mathbb{N} \mid S_i.rank = M_n\}$. We additionally extract $left = \{S_j \in S \mid j < i\}$ and $right = \{S_j \in S \mid j > i\}$ so that $left \cup x \cup right = S$. In this definition, the *pivot* element x is the smallest element in S that has the maximal rank among all elements in S . This process is then recursively applied to both $left$ and $right$, which are used to form the left and right subtrees of T respectively.

Algorithm 1: Recursive zip tree construction

```

1 map( $S$ ):
2   if  $S = \emptyset$  then return null
3    $M_n \leftarrow \max\{s.rank \mid s \in S\}$ 
4    $i \leftarrow \min\{i \in \mathbb{N} \mid S_i.rank = M_n\}$ 
5    $left \leftarrow \text{map}(\{S_j \in S \mid j < i\})$ 
6    $right \leftarrow \text{map}(\{S_j \in S \mid j > i\})$ 
7   return ( $key : S_i.key, rank : M_n, left, right$ )

```

Informally, the element x is chosen such that it has higher rank than all elements in $left$ and equal or higher rank than all elements in $right$. Furthermore, x itself is chosen so that its *key* is greater than all keys in $left$ and less than all keys in $right$, maintaining the sorted property of the set, and maintaining the invariants required by our zip tree definition.

Taking the zip tree from fig. 1, we have the sorted set of $(key, rank)$ tuples $S = \{(2, 1), (3, 2), (5, 1), (7, 3), \dots, (59, 1), (61, 2), (67, 3), (71, 2)\}$, which is split (in order) at the pivot elements 7, 3, 31, 13, 23, 17, 53, 41, 43, 67, and 61 to form the zip tree.

3 Geometric trees

This recursive definition reveals an interesting property of zip trees: *rank collisions*. By construction, rank collisions result from sequences of items with equal rank being arranged in a linked list of right children. We say a sequence $Q = q_0 \prec q_1 \prec \dots \prec q_k$ of items is *colliding* in a superset $S \supseteq Q$ if all items in Q have the same rank, and there is no item $s \in S$ of greater rank with $q_0 \prec s \prec q_k$. We can see this more explicitly in fig. 1, where the linked list $7 \rightarrow 31 \rightarrow 53 \rightarrow 67$ is formed by a colliding set of maximal rank items. In [3], they refer to these colliding sets as *r1-rank groups*, and show that the expected number of nodes in

an r1-rank group is well defined and predictable, a property that we will exploit in the design of our geometric tree.

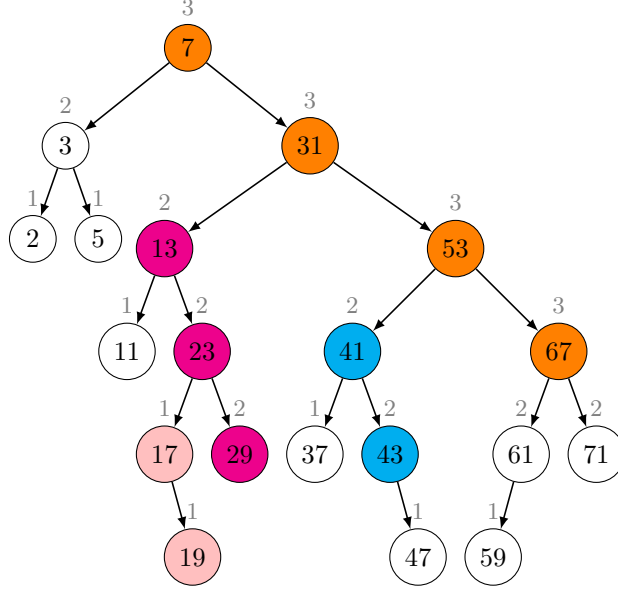


Figure 1: Zip tree with colliding linked lists of rank **three**, **two**, **two**, and **one**. Ranks are given as gray labels.

Let S be a set, let $T = (t_0, t_1, \dots, t_{|T|-1})$ be the items of maximal rank in S , and let $\mathfrak{S}$ be a data structure for representing sets. We define a *geometric search tree* recursively as either an empty tree, or a non-empty tree such that it has a root *node* = $(rank, items, next)$ represented as a (named) tuple, with $rank \in \mathbb{N}$, and where *items* is an instance of $\mathfrak{S}$ containing (named) tuples of type $(key, value)$, such that $key \in U$, and both *value* and *next* are valid subtrees.

We construct the geometric search tree by generalizing alg. 1 from using a single pivot item at each step of the recursion, to using *all* items of maximal rank as pivots in each step. As such we form a geometric search tree on S with $\mathfrak{S}$, using an instance of $\mathfrak{S}$ containing the items in T in ascending order together with their left subtrees to form the $(key, value)$ tuples in *items*, where the left subtree (*value*) of item t_i (*key*) is the geometric tree on $\{s \in S \mid t_{i-1} \prec s \prec t_i\}$ using $\mathfrak{S}$. We additionally reference the *next* right subtree (which may be *null*) as the geometric tree on $\{s \in S \mid t_{|T|-1} \prec s\}$ using $\mathfrak{S}$. See alg. 2 (which is a generalization of alg. 1) for an implementation of this recursive mapping function, which takes as input a sorted set S and can be carried over $\mathfrak{S}$.

Fig. 2 shows the geometric tree that is formed using the same set of values used to produce the zip tree in fig. 1. Note that colliding rank groups in the zip tree are now represented as sets of *items* in the geometric tree, with the left

Algorithm 2: Recursive geometric tree construction

```

1 map( $\mathfrak{S}, S$ ):
2   if  $S = \emptyset$  then return null
3    $M_n \leftarrow \max \{s.rank \mid s \in S\}$ 
4    $I \leftarrow \{i \in \mathbb{N} \mid S_i.rank = M_n\}$ 
5    $items \leftarrow \mathfrak{S}(\emptyset)$ 
6   for  $i \leftarrow 0$  to  $|I| - 1$  do
7      $(l, h) \leftarrow ((I_{i-1} + 1) \vee 0, I_i)$ 
8      $key \leftarrow S_h.key$ 
9      $value \leftarrow \text{map}(\{S_j \in S \mid l \leq j < h\})$ 
10     $items \leftarrow \text{push}(items, (key, value))$ 
11   $next \leftarrow \text{map}(\{S_j \in S \mid j > I_{|I|-1}\})$ 
12  return  $(rank : M_n, items, next)$ 

```

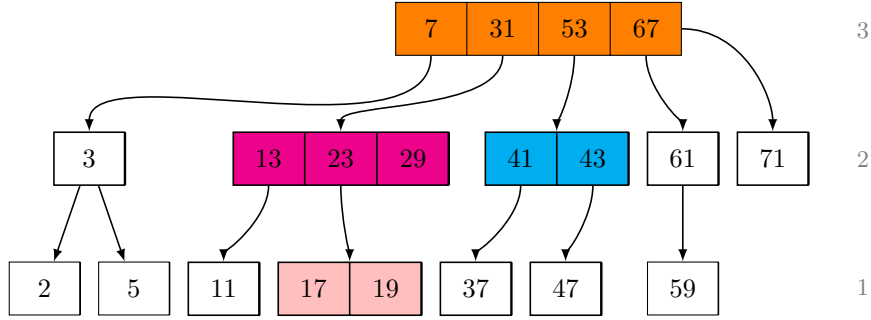


Figure 2: Geometric tree version of the zip tree from fig. 1. Ranks are labeled on the right in gray.

subtrees of the items forming the child pointers, and the *next* pointer identified via the rightmost pointer. The ranks are labeled on the right in gray, and the nodes are colored in the same way as the colliding rank groups in fig. 1.

4 Mutations

Zip trees partly derive their name from the fact that they use a pair of operations called “zipping” and “unzipping” (rather than tree rotations) to mutate their structure. Zipping involves combining the right spine of a left subtree and the left spine of a right subtree into a single path, by merging them from top to bottom in non-increasing rank order, breaking ties in favor of the smaller key (see fig. 3). Unzipping is essentially the opposite operation, where a path is split into two spines, one for the left subtree and one for the right subtree.

To insert a new node x into a zip tree, we start by searching for x in the tree until reaching the node y that x will replace, namely the node y such that

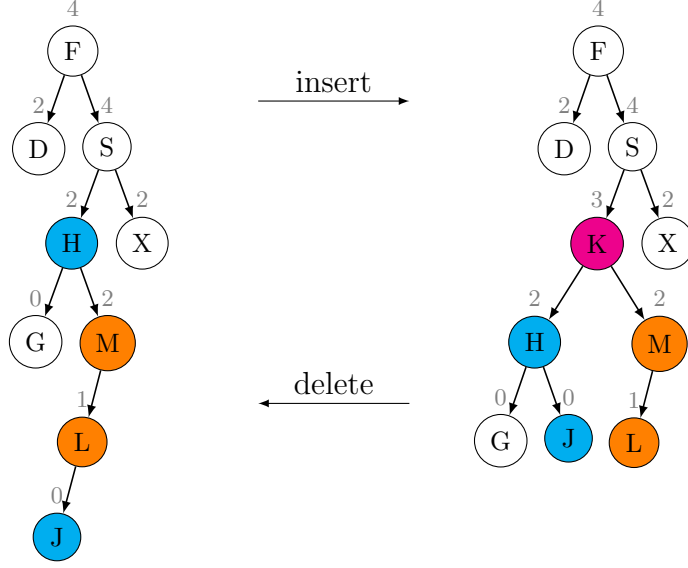


Figure 3: Mutating a zip tree via zipping/unzipping search path spines (after [8]). When removing K , the topmost node of the path must be H because it is the smallest value with the given rank (both H and M have rank 2). The remaining nodes on the path ($M \rightarrow L \rightarrow J$) are interleaved by rank and value.

$y.rank \leq x.rank$, with strict inequality if $y.key < x.key$. From y , we follow the rest of the search path for x , unzipping it by splitting it such that the left side contains nodes with key less than $x.key$ and the right side contains nodes with key greater than $x.key$. We then join the left and right sides as children of x to form a single sub-tree rooted at x . Walking back up the search path, we update any pointers to maintain our purely functional design.

Deletion is essentially the inverse of insertion. To delete a node x , we again do a search to find the node x where $x.key = key$. From x , we simply **zip** the left and right children to form a single path merging them, dropping x in the process.

4.1 Zip and unzip

Given we are interested in purely functional variants of our data structures, we redefine the algorithms provided in [8] to be purely functional. We start with the **zip** function in alg. 3, which is a simple recursive function that is nearly identical to its original recursive design. We simply replace the in-place updates with new nodes that are created and returned. The **zip** function takes two nodes, *left* and *right*, and returns a new node that is the result of merging the two nodes. The only new notation is the use of the (...) “spread” operator, which is used here to denote expanding and extracting the fields of a named

tuple.

Algorithm 3: Purely functional zip

```

1 zip(left, right):
2   if left = null then return right
3   if right = null then return left
4   if left.rank < right.rank then
5     left ← zip(left, right.left)
6     return (...right, left)
7   else
8     next ← zip(left.next, right)
9     return (...left, next)

```

For completeness, we also provide a purely functional **unzip** function in alg. 4. The **unzip** function is a recursive function that takes a key and a root node and returns a triple that contains the *left* and *right* subtrees, along with the target *node*.

Algorithm 4: Purely functional unzip

```

1 unzip(key, root):
2   if root = null then return [null, null, null]
3   if root.key = key then
4     rank ← root.rank
5     return [root.left, (key, rank), root.right]
6   else if root.key < key then
7     [next, node, right] ← unzip(key, root.right)
8     left ← (...root, right : next)
9     return [left, node, right]
10  else
11    [left, node, prev] ← unzip(key, root.left)
12    right ← (...root, left : prev)
13    return [left, node, right]

```

4.2 Extending zip and unzip

Like with zip trees, insertion and deletion in a geometric tree are done via zipping and unzipping. In order to leverage the geometric tree structure, we need to extend the **zip** and **unzip** functions to handle the additional keys and children that are present in the geometric tree. The **zip** function is shown in alg. 5, and the **unzip** function is shown in alg. 6.

Algorithm 5: Zip function for geometric trees

```

1 zip(left, right):
2   if left = null then return right
3   if right = null then return left
4   if left.rank = right.rank then
5     [rights, child] ← shift(right.items)
6     value ← zip(left.next, child.value)
7     inner ← (key : child.key, value)
8     items ← join(push(left.items, inner), rights)
9     return norm(...right, items)
10  else if left.rank < right.rank then
11    [rights, child] ← shift(right.items)
12    value ← zip(left, child.value)
13    items ← unshift(rights, (key : child.key, value))
14    return norm(...right, items)
15  else
16    next ← zip(left.next, right)
17    return norm(...left, next)

```

The **shift**, **unshift**, **pop**, and **push** functions are the usual functional versions for operations on a deque (note that *items* can be treated as a catenable deque), and **join** is used to efficiently concatenate two *items* sets. Similarly, **split** is used to efficiently split the *items* set at the smallest item where $item.key \geq key$. The **norm** function simply replaces any node that has an empty *items* set with its own *next* pointer (which may also be *null*). This ensures that the tree invariants are maintained and we do not end up with “dangling” nodes:

$$\text{norm}(node) = \begin{cases} node.next & \text{if } node.items = \emptyset \\ node & \text{otherwise} \end{cases}$$

As is clear from alg. 5, the zipping process for geometric trees is almost identical to zipping in the binary case (with some modifications to handle the additional keys/children). The main difference is that we now have to handle the case where the ranks of the two nodes are equal (lines 4-9). In this case, we take the leftmost item from the right node and the rightmost (i.e., *next*) item from the left node, and zip/merge them into a single inner node. We then join this inner node with rest of the *items* from the left and right nodes, and return the result. This is done in a way that ensures that the *items* are interleaved by rank and value, and that the resulting *items* remains sorted.

Similarly, alg. 6 demonstrates that unzipping in a geometric tree is equivalent to unzipping in the binary case (e.g., the case where the key is found in the items set of the root node (lines 8-12) is effectively unchanged). The main differences here are that 1) if the key is not found in the *items* set, we have

Algorithm 6: Unzip function for geometric trees

```
1 unzip(key, root):
2   if root = null then return [null, null, null]
3   [lefts, node, rights] ← split(root.items, key)
4   if node then
5     if node.key = key then
6       left ← norm(...root, items : lefts, next : node.value)
7       right ← norm(...root, items : rights)
8       item ← (key : node.key, rank : root.rank)
9       return [left, item, right]
10    [next, item, value] ← unzip(key, node.value)
11    left ← norm(...root, items : lefts, next)
12    items ← unshift(rights, (key : node.key, value))
13    right ← norm(...root, items)
14    return [left, item, right]
15  [next, item, right] ← unzip(key, root.next)
16  left ← norm(...root, next)
17  return [left, item, right]
```

to recursively search for it in the *next* subtree (lines 4-7), and 2) we no longer have two separate *left* and *right* subtrees, but rather a single *items* set that we need to split (lines 13-18).

4.3 Insert and delete

As with zip trees, an insertion or deletion in a geometric tree requires a search plus an **unzip** and/or a **zip**. Fig. 4 provides a visual representation of the **zip/unzip** process which joins/splits (gray dotted line) the two sub-trees rooted at A and B in 2 steps.

Alg. 7 provides an implementation of **insert** for geometric trees which *implicitly* uses search and unzipping to mutate the tree structure. This is a relatively direct “port” of the original zip tree **insert** function from [8].

The **delete** function is significantly simpler than the **insert** function, due in part to the fact that we are able to leverage the **zip** function from alg. 5 directly, whereas **insert** requires a more complex set of operations to *implicitly* unzip and manage the splits on the way “back up”.

5 Analysis of geometric trees

In this section, we analyze the geometric tree data structure. Here we are interested in its expected structure and performance under **zip**- and **unzip**-based mutations. To analyze the performance of the geometric tree, we need to

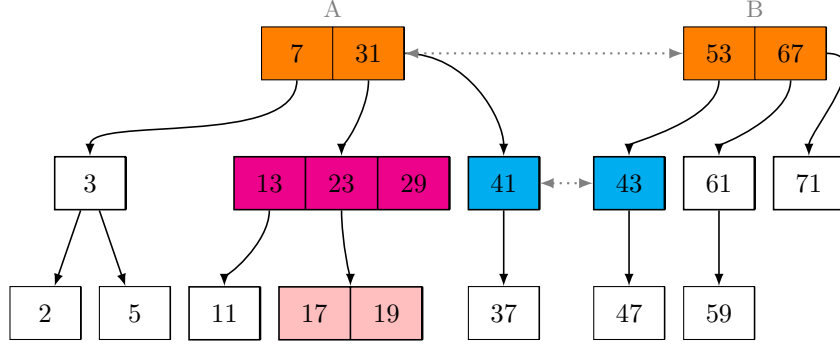


Figure 4: Zipping and unzipping the geometric tree from fig. 2 at 41 requires two steps. Sub-trees A and B are labeled in gray, and the dotted lines represent joins/splits.

bound its depth (i.e., the maximum rank), understand the distribution on node item set size, and understand how the node item set size distribution impacts the time to perform mutating operations on the trees.

5.1 Ranks

As in previous sections, let $S = \{s_1, s_2, \dots, s_n\}$ be a sorted set, where each element $s_i = (key, rank)$ has an associated *key* and *rank*. The *rank* is a random variable drawn from a (truncated) geometric distribution with parameter $p \in (0, 1)$, truncated at some maximum rank β . Here we define the geometric distribution $\mathcal{G}(p)$ to be the discrete probability distribution of the number X of biased coin flips (up to $\beta - 1$) required to obtain the first success (i.e., “heads”). As in [5], if all $\beta - 1$ coin flips turn up “tails,” then the rank is set to β . The distribution over ranks is thus parametrized by the probability of tails $q = 1 - p$ and a bound β on the maximum allowable level. The probability of a given rank is thus [4]:

$$\Pr(X = k) = \begin{cases} pq^{k-1} & \text{if } 1 \leq k < \beta \\ q^{\beta-1} & \text{if } k = \beta \\ 0 & \text{if } k \leq 1 \text{ or } k > \beta \end{cases}$$

For large β , we can ignore the effect of truncation, and we get the usual expected value $E[X] = 1/p = 1/(1-q)$. For smaller β , given that the probability for not getting a success in the first $k - 1$ trials is q^{k-1} , and the fact that we “stop” our biased coin flips after β trials, we reduce the expected number of trials by $1/(1-q)$ with probability $q^{\beta-1}$. This leads to the (truncated) expected value $E[X] = (1 - q^{\beta-1})/(1 - q)$.

Algorithm 7: Insert for geometric trees

```
1 insert(item, root):
2   if root = null then return item
3   [lefts, node, rights]  $\leftarrow$  split(root.items, key)
4   if node  $\neq$  null then
5     left  $\leftarrow$  insert(item, node.value)
6     if left.rank < root.rank then
7       inner  $\leftarrow$  (key : node.key, value : left)
8       items  $\leftarrow$  join(push(lefts, inner), rights)
9       return norm((...root, items))
10    else if left.rank = root.rank then
11      inner  $\leftarrow$  (key : node.key, value : left.next)
12      items  $\leftarrow$  join(push(join(lefts, left.items), inner),
13        rights)
14      return norm((...root, items))
15    else
16      r  $\leftarrow$  unshift(rights, (key : node.key, value : left.next))
17      next  $\leftarrow$  norm((...root, items : r))
18      last  $\leftarrow$  last(left.items)
19      value  $\leftarrow$  norm((...root, items : lefts, next : last.value))
20      items  $\leftarrow$  push(pop(left.items), (key : last.key, value))
21      return norm((...left, items, next))
22  else
23    right  $\leftarrow$  insert(item, root.next)
24    if right.rank < root.rank then
25      return norm((...root, items : lefts, next : right))
26    else if right.rank = root.rank then
27      items  $\leftarrow$  join(lefts, right.items)
28      return norm((...root, items, next : right.next))
29    else
30      last  $\leftarrow$  last(right.items)
31      value  $\leftarrow$  norm((...root, items : lefts, next : last.value))
32      items  $\leftarrow$  push(pop(right.items), (key : last.key, value))
33      return norm((...right, items))
```

5.2 Height

A geometric tree T is parameterized by a tuning parameter $\gamma \in \{n \in \mathbb{N}_+ \mid n \geq 2\}$, which is used to control the expected height/depth of T , and the expected *items* set size (i.e., number of children) of each node $g \in T$. Let the target capacity N be some anticipated maximum number of keys in T . Fix

Algorithm 8: Delete for geometric trees

```

1 delete(key, root):
2   if root = null then return null
3   [lefts, node, rights]  $\leftarrow$  split(root.items, key)
4   if node  $\neq$  null then
5     if node.key = key then
6       left  $\leftarrow$  norm(...root, items : lefts, next : node.value)
7       right  $\leftarrow$  norm(...root, items : rights)
8       return zip(left, right)
9     value  $\leftarrow$  delete(key, node.value)
10    inner  $\leftarrow$  (key : node.key, value)
11    items  $\leftarrow$  join(push(lefts, inner), rights)
12    return norm(...root, items)
13  else
14    next  $\leftarrow$  delete(key, root.next)
15    return norm(...root, next)

```

a parameter γ , and let $q = 1/\gamma$, and $\beta = \lceil \log_\gamma N \rceil + 2$. Then, $g.rank$ is a (truncated) geometric random variable with parameter $p = 1 - q = 1 - 1/\gamma$, truncated at β .

The root rank, $M_n = \max\{g.rank \mid g \in T\}$ is the maximum of n (independent) samples of the (truncated) geometric distribution. By construction, this is at most $\beta = \lceil \log_\gamma N \rceil + 2$. For $n \ll N$, we can bound M_n as the maximum rank of any $g \in T$ via the union bound. Note that for given $k \leq \beta$ the probability that the rank of a node g is at least k is at most q^{k-1} (i.e., $\Pr[g.rank \geq k] \leq q^{k-1}$). This implies that the probability that there exists some g such that $g.rank \geq k$ is $\Pr[\exists g, g.rank \geq k] \leq nq^{k-1}$, and so for some positive constant c , $\Pr[M_n \geq (c+1) \log_\gamma n] \leq n^{-c}$ [4].

Even for very large $\beta \rightarrow \infty$, the tail probability contribution is relatively small, and can be estimated via the simplification of the geometric series for values of $k > \lceil \log_\gamma n \rceil$:

$$\sum_{k=\lceil \log_\gamma n \rceil + 1}^{\infty} np^{k-1} = np^{\lceil \log_\gamma n \rceil} \leq np^{\log_\gamma n} = \frac{1}{1-p} = \frac{1}{q}$$

which is q , and implies that even without truncation, the expected max rank $E[M_n]$ is $\lceil \log_\gamma n \rceil$, and is at most $\lceil \log_\gamma n \rceil + 1/q$ (i.e., $\log_\gamma n + \mathcal{O}(1)$), with high probability. The same expectation for M_n is given by [7, eq. (2.6) and (2.12)] (among others), indicating that the expected value of M_n is

$$E[M_n] = - \sum_{k=1}^n (-1)^k \binom{n}{k} \frac{1}{1 - q^k},$$

which is asymptotically equal to $\log_\gamma n + \mathcal{O}(1)$.

By construction, the height of a geometric tree $H(T)$ is less than or equal to its maximum rank M_n . Like zip trees, geometric trees “compress” their depth where ranks are missing or skipped. While we don’t bother with tight bounds on this compressed height, [1] provides an expectation on the number of distinct ranks $E[d_n]$ in a geometrically distributed random sample, which could provide tighter bounds (though as they point out, the expected number of missing values in the range 1 up to $E[M_n]$ is asymptotically given by $E[M_n] - E[d_n] \approx 1 - \log_\gamma(\gamma - 1)$, which makes little difference for $\gamma > 2$).

In any case, $E[H(T)] \leq E[M_n] \leq \lceil \log_\gamma n \rceil + \gamma$ with high probability, and at most β with certainty when truncation is applied. As such, the expected height of a geometric tree is roughly $\mathcal{O}(\log n)$.

5.3 Nodes

Given the geometric distribution of (independent) ranks over input values $s \in S$, the expected number of values with $rank = k$ is about γ times the expected number of values at rank $k + 1$ (i.e., $\Pr[s.rank > k | s.rank \geq k] = q$). Let T be a geometric tree. Recall that each node $g \in T$ also has a $rank$, derived from its $items$. Based on our top-down recursive definition, we can think of nodes within T as disjoint subsets formed by splitting the (sorted) items of rank k at values of rank $k + 1$. Furthermore, the number of nodes with a given rank k is a direct function of the number of nodes with rank $k + 1$. This implies that the *size* of the *items* set of a node g , $|g.items|$ is itself a random variable drawn from a geometric distribution with parameter $1/\gamma$ and support $[n] = \{1, \dots, n\}$. The expected value is roughly γ , and $|g.items|$ is at most $c = \lceil \log_\gamma n \rceil$ times the expected value with probability at least $1 - (1 - q)^{c\gamma}$. In other words, $\Pr[|g.items| \geq c\gamma] \leq (1 - q)^{c\gamma}$, and so $|g.items|$ is close to expectation with high probability.

With the above in mind, the total expected number of nodes in a geometric tree is $E(|T|) = n/\gamma + 1 = nq + 1$. While this expectation is intuitive, it can be estimated more directly as the sum of the expected number of nodes at each rank $k \in \{1 \dots \beta - 1\}$, plus a root node. Since the expected number of nodes at rank k is equal to the expected number of *values* having rank $k + 1$, the total expected number of nodes in a geometric tree where $\beta \rightarrow \infty$ is

$$\sum_{k=1}^{\infty} n(1 - q)q^k + 1 = n(1 - q) + 1 \sum_{k=1}^{\infty} q^k,$$

and recognizing that the rightmost sum is $\frac{q}{1 - q}$, substituting this into the original expression, we get

$$n(1 - q)\frac{q}{1 - q} + 1 = nq + 1.$$

To derive bounds, we leverage the fact that the sum of n i.i.d geometrically distributed random variables with expectation $q = 1/\gamma$ is a negative binomially

distributed random variable $Y(n, q)$. Following the concentration inequality for the sum of geometric random variables from [2], we have $E[Y(n, q)] = nq$, and for $c > 1$, $\Pr[Y > (1 + c)nq] \leq \exp(\frac{-c^2qn}{2(1+c)})$. This implies that the expected total number of nodes in a geometric tree with n keys is less than $nq + 1$ with high probability, and thus the expected space complexity of a geometric tree is roughly $\mathcal{O}(n)$.

5.4 Search

By construction, the nodes of the geometric tree partition the input set S into a hierarchy of disjoint partitions, such that for a given g we have that for all $(k, v) \in g.items$, $v.rank < g.rank$, and for all pairs $(k', v') \in v.items$, we have $k' < k$ and $v'.rank < v.rank$. Specifically, if $g \in T$ represents a subset S_g of the input set S , then each child of g represents a disjoint subset of S_g , and the union of all these subsets equals S_g . This structure ensures that each element of S is uniquely represented in *one* of the nodes of the tree, maintaining the integrity of the partitioning throughout the hierarchy. As such, a search for a given key will visit (or access) at most one node per rank, and will reduce the search space by a factor of roughly γ at each step. Given that the expected height $H(T)$ of a geometric tree T is close to $\lceil \log_\gamma n \rceil$ with high probability, the expected time complexity of a search operation is $\mathcal{O}(\log n)$.

5.5 Insert and delete

As with zip trees, an insertion or deletion in a geometric tree requires a search plus an **unzip** and/or a **zip**. The cost of a **zip** or **unzip** is proportional to the *depth* of the resulting/initial sub-tree at g , which itself is inversely proportional to the rank of the sub-tree rooted at g . For instance, for node g with $g.rank = k$, the depth is equal to $M_n - k - 1$, with expectation and bounds similar to those for M_n itself, shifted by k (i.e., for some positive constant c , $\Pr[M_n - k - 1 \geq (c + 1)(\log_\gamma n - k - 1)] \leq n^{-c}$).

In the case of **delete**, when an item y with a given rank k is removed, we require one “join” operation (step) per rank $< k$ during the **zip** (i.e., the cost is equal to the depth of the sub-tree rooted at the node g such that $y \in g$). Thus the expected number of join operations during a **delete** is equal to the expected rank/height of a node, which as we have seen before is $E[X] = 1/p = 1/(1 - q)$ or $E[X] = (1 - q^{\beta-1})/(1 - q)$ when truncation at β is used. Combining this with the expected number of steps in a search, we get that the expected cost of deletion is $\mathcal{O}(\log n)$ with high probability. The **insert** operation is similar, with the combined cost of a search and an **unzip** being proportional to the depth of the resulting sub-tree rooted at g where the item y is inserted. The expected cost of insertion is thus also $\mathcal{O}(\log n)$ with high probability.

A Extensions

In this section, we outline some possible extensions to the geometric tree data structure. These include adding support for representing other high-level data structures, such as sequences, and supporting alternative data layouts, such as storing data pointers only at the leaf nodes of the tree.

We can very easily (re)define both `insert` and `delete` operations using a functional design that works strictly based on `zip` and `unzip`.

Algorithm 9: Insert and delete functions based on (un)zip

```

1 insert(item, root):
2   [left, node, right]  $\leftarrow$  unzip(x.key, root)
3   return zip(zip(left, item), right)

1 delete(key, root):
2   [left, node, right]  $\leftarrow$  unzip(key, root)
3   return zip(left, right)

```

A.1 Sequences

In our discussion of geometric trees, our focus has been using them to represent abstract container types with an explicit ordering, such associative containers (sets, maps, etc.). However, sequences are a natural next step for our discussion, as they are the simplest form of an implicitly ordered collection. Sequences are also a common building block for more complex data structures, such as lists, strings, and arrays. We consider a sequence to be an ordered collection of values, where each value is identified by its position (or index) in the sequence.

Semantically, sequences are similar to our treatment of sorted sets, as they are both ordered collections of values. However, sequences differ from sets in that they allow duplicate values and are indexed by position rather than by value. This simplifies their treatment to some degree, as we are able to avoid potentially expensive comparisons between values. However, it also introduces some complexity, as we must now explicitly (rather than implicitly) consider the ordering of elements in the sequence.

In practice, we are able to reuse much of the machinery we developed for the general geometric tree structure. As in [6], we assume we can efficiently compute (and cache) the `length` of a tree T , which is simply the number of elements in the sequence represented by T . This is assumed to be a constant-time function. Furthermore, we redefine the `split` function used over our internal set structure $\mathfrak{S}$ to accept any arbitrary predicate function to identify the split, rather than just a key. For example, let $g \in T$ be a node in the tree, and let m be a given offset. We define a predicate function $f : \{1, 2, \dots, |g|\} \rightarrow \{true, false\}$ as follows:

$$f(i) = \begin{cases} \text{true} & \text{if } \sum_{j=1}^i \text{length}(g.items_i) \geq m \\ \text{false} & \text{otherwise} \end{cases}$$

This function takes an index i and returns *true* if the cumulative length of the first i values in $g.items$ is at least m , and *false* otherwise. By simply changing how we operate on $\mathfrak{S}$, the bulk of the **unzip** function operates as before, though some adjustments are needed to account for the remainder of the offset m at each level of recursion.

Algorithm 10: Queries on sequences using geometric trees

```

1 first( $m, root$ ):
2   [ $left, node, right$ ]  $\leftarrow$  unzip( $m, root$ )
3   return  $left$ 

1 last( $m, root$ ):
2   [ $left, node, right$ ]  $\leftarrow$  unzip( $\text{length}(root) - m, root$ )
3   return  $right$ 

```

With these minor changes, we are able to define a series of efficient queries and operations on sequences that are defined strictly in terms of the **zip** and **unzip** functions.

Algorithm 11: Basic operations on sequences using geometric trees

```

1 insert( $m, item, root$ ):
2   if  $root = \text{null}$  or  $i > \text{length}(root)$  or  $i < 0$  then
3     if  $i = 0$  then return  $item$ 
4     else return  $\text{null}$ 
5    $head \leftarrow \text{first}(m, root)$ 
6    $tail \leftarrow \text{shift}(item, \text{last}(\text{length}(root) - m, root))$ 
7   return zip( $head, tail$ )

1 delete( $m, item, root$ ):
2    $head \leftarrow \text{first}(m, root)$ 
3    $tail \leftarrow \text{last}(\text{length}(root) - m - 1, root)$ 
4   return zip( $tail, heads$ )

```

Notice also that this set of operations (along with obvious implementations of **join** and **split**) is sufficient to satisfy the deque interface we require for our internal set data structure, which would allow us to bootstrap the geometric tree structure with an appropriately parameterized version of $\mathfrak{S}$.

Algorithm 12: Further operations on sequences using geometric trees

```
1 push(item, root):  
2   return zip(root, item)  
  
1 shift(item, root):  
2   return zip(item, root)  
  
1 pop(root):  
2   [left, node, right]  $\leftarrow$  unzip(length(root) - 1, root)  
3   return [left, node]  
  
1 unshift(root):  
2   [left, node, right]  $\leftarrow$  unzip(1, root)  
3   return [right, node]
```

B Tail recursion

The **zip** and **unzip** functions can be made tail-recursive by using the continuation passing style (CPS) transformation. This is a style of programming in which the function does not return a value *per se*, but instead takes an extra argument — the continuation — which is a function that represents the rest of the computation. This allows the function to return immediately, and the rest of the computation is represented by the continuation function. In our case, we use a continuation to optimize for tail call optimization (TCO). The function provided in alg. 5 can be modified to use TCO as shown in alg. 13, where *id* is the identity function, and $(\dots args) \rightarrow \dots$ defines a lambda-style continuation function.

Similarly, the function provided in alg. 6 can be modified to use TCO as shown in alg. 14.

Algorithm 13: Tail recursive zip function

```
1 zip(left, right, cont = id):
2   if left = null then return cont(right)
3   if right = null then return cont(left)
4   if left.rank = right.rank then
5     [rights, child] ← unshift(right.items)
6     return zip(left.next, child.value, (value) →
7       inner ← (key : child.key, value)
8       items ← join(push(left.items, inner), rights)
9       return cont(norm(...right, items)))
10  )
11  else if left.rank < right.rank then
12    [rights, child] ← unshift(right.items)
13    return zip(left, child.value, (value) →
14      items ← shift(rights, (key : child.key, value))
15      return cont(norm(...right, items)))
16  )
17  return zip(left.next, right, (next) →
18    return cont(norm(...left, next)))
19  )
```

Algorithm 14: Tail recursive unzip function

```
1 unzip(key, root, cont = id):
2   if root = null then return cont([null, null, null])
3   [lefts, node, rights] ← split(root.items, key)
4   if node = null then
5     return unzip(key, root.next, (next, n, right) →
6       left ← norm(...root, next)
7       return cont([left, n, right])
8     )
9   else if node.key = key then
10    left ← norm(...root, items : lefts, next : node.value)
11    right ← norm(...root, items : rights)
12    n ← (key : node.key, rank : root.rank)
13    return cont([left, n, right])
14  return unzip(key, node[1], (next, n, value) →
15    left ← norm(...root, items : lefts, next)
16    items ← shift(rights, (key : node.key, value))
17    right ← norm(...root, items)
18    return cont([left, n, right])
19  )
```

References

- [1] Margaret Archibald, Arnold Knopfmacher, and Helmut Prodinger. The number of distinct values in a geometrically distributed sample. *European Journal of Combinatorics*, 27(7):1059–1081, 2006. Eurocomb '03 - Graphs and Combinatorial Structures.
- [2] Daniel G Brown. How i wasted too long finding a concentration inequality for sums of geometric variables, 2011.
- [3] Ofek Gila, Michael T Goodrich, and Robert E Tarjan. Zip-zip trees: Making zip trees more balanced, biased, compact, or persistent. In *Algorithms and Data Structures Symposium*, pages 474–492. Springer, 2023. <https://arxiv.org/pdf/2307.07660.pdf>.
- [4] Daniel Golovin. The b-skip-list: A simpler uniquely represented alternative to b-trees. *arXiv preprint arXiv:1005.0662*, 2010. <https://arxiv.org/pdf/1005.0662.pdf>.
- [5] William Pugh. Skip lists: a probabilistic alternative to balanced trees. *Communications of the ACM*, 33(6):668–676, 1990. <https://dl.acm.org/doi/pdf/10.1145/78973.78977>.
- [6] William Pugh and Tim Teitelbaum. Incremental computation via function caching. In *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 315–328, 1989. <https://dl.acm.org/doi/pdf/10.1145/75277.75305>.
- [7] Wojciech Szpankowski and Vernon Rego. Yet another application of a binomial recurrence order statistics. *Computing*, 43(4):401–410, 1990.
- [8] Robert E Tarjan, Caleb Levy, and Stephen Timmel. Zip trees. *ACM Transactions on Algorithms (TALG)*, 17(4):1–12, 2021. <https://arxiv.org/pdf/1806.06726.pdf>.